

Chapter 1

Introduction to Compilers

From source code to running machine code

Compiler Construction Module

Based on Lecture 1

1 What a Compiler Is, and Why We Need One

1.1 Motivation and context

You write programs in a language designed for *people*: named variables, loops, types, indentation. The processor understands none of that. It executes fixed-width binary instructions that load registers, add numbers, and jump to addresses. Somebody has to bridge the gap, and doing it by hand is slow and error-prone. That is exactly what programmers did in the early 1950s, and it is why Grace Hopper's A-0 system and John Backus's FORTRAN compiler (1957) were regarded as a breakthrough: FORTRAN's output was nearly as fast as hand-written assembly, so people stopped writing assembly.

Think of a compiler as a professional translator working on a legal contract. The translation must preserve *meaning* exactly, the output must be in the idiom of the target reader (the CPU), and one mistranslated clause can be catastrophic. That last point is the theme of this course: a compiler that inserts bugs destroys your ability to trust any program at all.

Concept: A compiler is a translator

A **compiler** translates programs from a **source language** into a **target language** while preserving their meaning. In the classic case, the source is a high-level language designed for humans and the target is assembly or machine code designed to run efficiently on hardware. The translation need not happen in one hop. Java uses two compilers in sequence: `javac` turns Java into JVM bytecode ahead of time, and a JIT compiler turns bytecode into native code at run time.

History. FORTRAN (1957) took 18 person-years. Its optimizer was so good that sceptics who thought compilers could never match humans were silenced. Today's techniques make a working compiler a one-semester project.

1.2 Compilation versus interpretation

A compiler is not the only way to run a program. An **interpreter** simulates execution directly, statement by statement, with no separate translation step. Both approaches are correct; they differ in where the work happens.

Trade-offs: Compile or interpret?

Compiling

Translate once, then run fast target code on the hardware. At least an order of magnitude faster; an order of magnitude less energy and heat.

Translation costs time and effort up front.

Best when the program runs many times.

Interpreting

Simulate execution directly, statement by statement.

Much slower: software simulating hardware costs performance.

No separate step; edit and run immediately.

Best when the program runs once (scripts, exploration).

Rule of thumb: interpret if you will run it once; compile if you will run it many times. Modern systems blur the line: a JIT interprets first, then compiles the hot paths.

1.3 Source code versus target code

Source code is optimised for humans: it keeps the notions of language and structure, and it carries deliberate redundancy (you declare `i: int` even though the compiler could often guess) that helps catch programmer mistakes. Target code is optimised for the hardware: it is not meant to be readable, and it throws away both the redundancy and the high-level structure. Because the two languages are so different, showing that a compiler preserves meaning is hard.

Key insight. Source code carries *redundancy* (types, declarations) that lets the compiler catch mistakes. Target code discards it, which is exactly why target code is hard to reason about.

Warning / Gotcha: Correctness above all

Performance is why compilers exist, but **correctness matters more**. If the compiler can insert bugs, no test of your program tells you anything, because the bug may be in the tool. With mathematical descriptions of source and target languages, “correct” can be defined precisely; CompCert is a C compiler whose correctness is machine-checked in Coq. This course does not build proofs, but it describes every phase declaratively and precisely for the same reason: a precise specification gives you something an implementation can be checked *against*.

1.4 The course in one sentence

Each compiler phase is first described *declaratively* (precise specifications, inference rules) and then compiled down into efficient, clear imperative code. Along the way you will meet the theory that compilers motivated: automata, grammars, and a surprising amount of algorithms and data structures. The source language also shapes the compiler, so you will come away understanding programming languages better in general.

In the real world. “It seems to work” is not a specification. Every compiler bug report you will ever read starts with a program that seemed to work.

Summary

A compiler translates source to target while preserving meaning. Compiling costs effort once and runs fast forever; interpreting is cheap up front and slow per run. Source code is for people and carries redundancy; target code is for hardware and carries none. Correctness beats performance, always.

2 A Complete Example: Compiling `findIndex`

2.1 Conceptual overview

To see the flavour of what a compiler produces, here is a short function in the language used throughout the course. It returns the index of the first array element equal to its own 1-based position, or `-1`.

```
findIndex(a: int[]): int {  
  n: int = length a  
  i: int = 0  
  while i < n {  
    if a[i] == i + 1 {  
      return i  
    }  
    i = i + 1  
  }  
  return -1  
}
```

The compiler can emit two very different programs for this one source, both correct. The **unoptimised** version is a mechanical translation: every variable lives in a stack slot and is reloaded before each use. The **optimised** version keeps every variable in a register.

2.2 Technical specification: the two outputs

Unoptimised (stack-based)	Optimised (register-based)
<pre> findIndex: push rbp mov rbp, rsp mov [rbp-16], rdi ; a mov rax, [rbp-16] mov rax, [rax-8] ; length a mov [rbp-24], rax ; n mov qword ptr [rbp-32], 0 ; i = 0 L1: mov rax, [rbp-32] cmp rax, [rbp-24] jge L6 L4: mov rax, [rbp-16] mov rcx, [rbp-32] mov rax, [rax+8*rcx] ; a[i] mov rcx, [rbp-32] add rcx, 1 ; i+1 cmp rax, rcx jne L5 mov rax, [rbp-32] mov [rbp-8], rax ; result = i jmp L7 L5: mov rax, [rbp-32] add rax, 1 mov [rbp-32], rax ; i = i+1 jmp L1 L6: mov qword ptr [rbp-8], -1 L7: mov rax, [rbp-8] pop rbp ret </pre>	<pre> findIndex: mov rcx, [rdi-8] ; rcx = n, for good xor edx, edx ; rdx = i = 0 L1: cmp rcx, rdx ; i == n ? je L2 lea rax, [rdx+1] ; rax = i+1 cmp rax, [rdi+8*rdx] ; == a[i] ? mov rdx, rax ; i = i+1 jne L1 dec rax ; rax was i+1 ret L2: mov rax, -1 ret </pre>

2.3 Implementation deep-dive: reading the assembly

Read the two columns row by row against the source:

- **n = length a; i = 0.** Unoptimised: load **a** from its slot, load the length stored at **a-8**, store it to **n**'s slot, store 0 to **i**'s slot (four memory operations). Optimised: one load into **rcx**, and **xor edx,edx** zeroes **rdx** with no memory at all.
- **while i < n.** Unoptimised: reload **i**, compare against **n** in memory, **jge** **L6**. Optimised: **cmp rcx,rdx** directly. Note the optimiser proved **i** never exceeds **n**, so it tests **je** (equal) instead of **jge**.
- **if a[i] == i + 1.** Unoptimised: reload **a**, reload **i**, index, reload **i** again, add 1, compare. Optimised: **lea rax,[rdx+1]** computes **i + 1** without touching memory (**lea** is address arithmetic reused as an adder), then compares against **a[i]** directly.
- **return i.** The optimised code already holds **i + 1** in **rax**, so it simply decrements and returns. The increment **mov rdx,rax** was hoisted *before* the branch because it is harmless on both paths.
- **return -1.** Both versions store **-1** into the return register; the unoptimised one goes via a stack slot first.

Key insight. The optimised loop touches memory exactly once per iteration (the **a[i]** load). The unoptimised loop does it about nine times.

Every one of the 18 stack accesses in the unoptimised version disappears. This transformation, **register allocation**, is the single most important optimisation a compiler performs.

2.4 The assembler's job

A compiler usually does not finish the job; it emits assembly text and leaves the encoding to the **assembler**. Assembling the optimised code produces bytes, and symbolic labels become numeric offsets.

Offset	Machine code (hex)	Assembly
0:	48 8b 4f f8	mov rcx, [rdi-8]
4:	31 d2	xor edx, edx
6:	48 39 d1	cmp rcx, rdx
9:	74 11	je 0x1c
b:	48 8d 42 01	lea rax, [rdx+1]
f:	48 3b 04 d7	cmp rax, [rdi+8*rdx]
13:	48 89 c2	mov rdx, rax
16:	75 ee	jne 0x6
18:	48 ff c8	dec rax
1b:	c3	ret
1c:	48 c7 c0 ff ff ff ff	mov rax, -1
23:	c3	ret

Technique: See this yourself

```
gcc -O0 -S -masm=intel findindex.c -o unopt.s    # stack-based
gcc -O2 -S -masm=intel findindex.c -o opt.s      # register-based
gcc -O2 -c findindex.c && objdump -d -M intel findindex.o #
    bytes + labels
```

Or paste the C into <https://godbolt.org> and flip between -O0 and -O2.

In the real world. The `je 0x1c` is encoded as `74 11`: opcode `74` plus a *relative* displacement of `0x11` from the next instruction (`0xb + 0x11 = 0x1c`). Likewise `75 ee` jumps back by `-18` bytes. This is why moving code around is the assembler's job, not the compiler's.

Summary

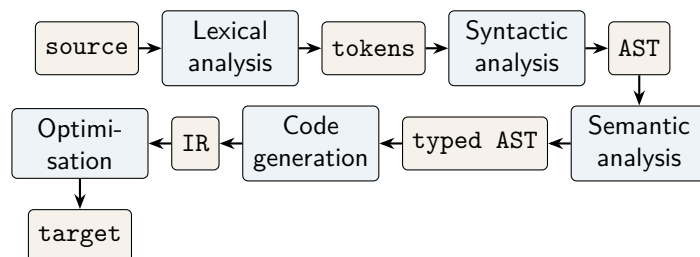
One source program, two valid translations. The unoptimised version is a faithful but slow mechanical transcription; the optimised version keeps every value in a register and touches memory once per iteration. The assembler then turns symbolic assembly into bytes and resolves labels into relative offsets.

3 Anatomy of a Compiler: The Phases

3.1 Motivation and context

It can seem magical that source code becomes correct, efficient machine code, and quickly. The trick is the same one used everywhere in engineering: *do not*

do it all at once. A sequence of intermediate languages sits between source and target, one per phase, each designed to make the *next* phase easy. A phase never has to understand the whole problem, only its own slice.



We trace one statement through every phase:

```
if (val >= 0) pos = val
```

3.2 Phase 1: lexical analysis

Mechanism: Lexer

Input: the program as a stream of bytes or characters. **Output:** a sequence of **tokens** (also called lexemes): keywords, identifiers, literals, operators, punctuation. Whitespace and comments are discarded.

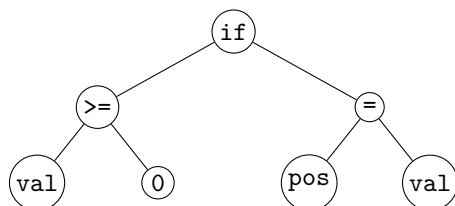
```
if ( val >= 0 ) pos = val
```

Nine tokens, with all spacing gone. The lexer is the subject of Chapter 2.

3.3 Phase 2: syntactic analysis (parsing)

Mechanism: Parser

Input: the token stream. **Output:** an **abstract syntax tree** (AST) capturing the program's structure and discarding details such as parentheses that do not affect meaning.

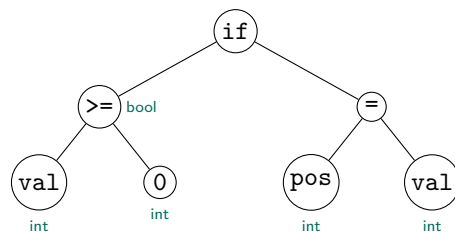


The nesting says it all: **if** has a condition child and a body child. Notice the parentheses are gone; they were only there to help the parser find the boundary of the condition.

3.4 Phase 3: semantic analysis

Mechanism: Semantic analyser / type checker

Input: the AST. **Output:** the same AST, *decorated* with extra information, chiefly the type of every expression, or a rejection if the program is not legal (undeclared name, type mismatch).



The checker looked up `val` and `pos` in a symbol table, found both are `int`, concluded `val >= 0` is `bool` (legal as an `if` condition), and that assigning `int` to `int` is fine.

3.5 Phase 4: code generation

Mechanism: Code generator

Produces an **intermediate representation** (IR) first, often as *abstract assembly* over a control-flow graph, and only later translates toward real target code.



The variables `val` and `pos` appear by name, as if the machine had unlimited registers. Both branches converge on the next statement, so the graph has no dangling edges.

3.6 Phase 5: optimisation

“Better” usually means faster, but it can mean less memory or less energy. Optimisation applies at almost every phase. Two steps on our example:

Key insight. Abstract assembly pretends there are infinitely many registers, one per variable. Deciding which *real* register each gets is postponed to optimisation.

+ register allocation	+ branch-free rewrite
<pre> cmp rax, 0 ; val in rax jl L1 mov rcx, rax ; pos in rcx L1: </pre>	<pre> cmp rax, 0 cmovge rcx, rax ; rcx=rax if >= 0 </pre>

Register allocation removes memory traffic for `val` and `pos` entirely. The conditional move `cmovge` removes the branch, which matters because a mis-predicted branch can cost 15–20 cycles on a modern core.

3.7 Front end and back end

The phases group naturally into two halves that talk through the IR:

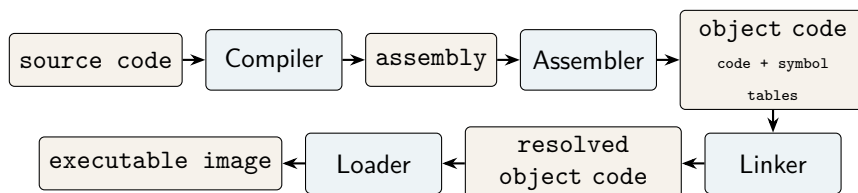
Front end	Back end
Lexical, syntactic, semantic analysis; code generation; machine-independent optimisation. Converts source to a <i>machine-independent</i> IR. Reusable across many target machines.	Instruction selection; machine-dependent optimisation; register allocation. Handles everything <i>machine-dependent</i> . Reusable across many source languages, if the IR is general enough.

In the real world. LLVM is exactly this split: Clang (C/C++), rustc, and Swift are front ends that emit LLVM IR; one shared back end targets x86, ARM, RISC-V, WebAssembly and more.

With m languages and n targets, this reduces $m \times n$ compilers to m front ends plus n back ends.

3.8 The bigger picture: from compiler to running program

The compiler produces assembly per source file; other tools finish the job.



The **assembler** encodes instructions and leaves unresolved references (to functions in other files or libraries) in a symbol table. The **linker** stitches object files together and resolves those references. The **loader** (part of the OS) maps the executable into memory and jumps to its entry point. This is why “undefined reference to `foo`” is a *linker* error, not a compiler error: the compiler happily accepted a declaration of `foo`, and only the linker discovered no object file defines it.

Summary

Five phases, each with its own intermediate language: characters $\rightarrow$ tokens $\rightarrow$ AST $\rightarrow$ typed AST $\rightarrow$ IR $\rightarrow$ target. The front end is machine-independent, the back end is language-independent, and the IR is the contract between them. After the compiler, the assembler, linker and loader finish the job.

4 Compilers and the Chomsky Hierarchy

4.1 Motivation and context

Every phase you just saw is a direct application of a result from computational theory. This is not a coincidence: much of that theory was invented *for* compilers. Knowing where each phase sits in the hierarchy tells you how expensive it must be, and, more usefully, why compilers are structured the way they are.

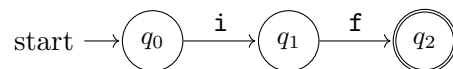
Concept: The Chomsky hierarchy, recalled

Languages fall into nested families, each defined by how much memory a machine needs to recognise it.

Level	Family	Recognising machine	Memory
Type 3	Regular	Finite automaton (DFA/NFA)	None beyond the current state
Type 2	Context-free	Pushdown automaton (PDA)	One unbounded stack
Type 1	Context-sensitive	Linear-bounded automaton (LBA)	Proportional to the input
Type 0	Recursively enumerable	Turing machine	Unbounded; may not halt

4.2 Type 3: the lexer is a finite automaton

Token classes (identifiers, numbers, keywords) are regular languages described by regular expressions. A lexer generator does exactly what you did by hand in the automata course: build an NFA from each regex, union them (regular languages are closed under union, which is why one DFA can recognise *all* token classes in a single pass), apply subset construction, minimise, and run the input through the DFA.



The keyword `if` accepted by a 3-state DFA.

A DFA reads each character once and does $O(1)$ work per character: lexing is $O(n)$.

4.3 Type 2: the parser is a pushdown automaton

Programming-language syntax is written as a context-free grammar, with productions such as

$$\begin{aligned} S &\rightarrow \text{if } (E) S & S &\rightarrow \text{id} = E \\ E &\rightarrow \text{id} \mid \text{num} \mid E \geq E \end{aligned}$$

A parser is a PDA whose stack holds the grammar symbols it still expects to see: *pushing is predicting, popping is matching*. Trace the stack while parsing `if ( val >= 0 ) pos = val`:

Input consumed so far	Stack (top first)	Meaning
<code>if (</code>	<code>E) S</code>	expect a condition, a paren, a body
<code>if (val >= 0</code>	<code>) S</code>	condition matched
<code>if (val >= 0)</code>	<code>S</code>	now expect the body
<code>... pos = val</code>	(empty)	done: accept

General context-free parsing is $O(n^3)$ (CYK, Earley). LL and LR parsing restrict the grammar to the *deterministic* slice of the context-free class, trading expressive power for linear-time, unambiguous parsing. Every production compiler makes that trade.

Key insight. Each symbol on the stack is a promise the parser still has to keep. That bookkeeping is exactly what a PDA's stack does.

4.4 Type 1: why semantic analysis is not a bigger grammar

“`x` must be declared before it is used” and “both sides of `=` must have the same type” are cross-referencing constraints that no context-free grammar can express. The classic witness is $\{a^n b^n c^n\}$, which needs two counts tracked simultaneously; matching every use of a variable to its one declaration is the same shape of problem: unbounded, simultaneous bookkeeping.

A context-sensitive grammar could express it, but LBA membership is PSPACE-complete, far too slow to run on every compile. So compilers cheat, productively:

1. parse with a context-free grammar (fast, linear time);
2. then walk the tree with a **symbol table** and **type environment**, checking everything the grammar alone could not.

Semantic analysis is a second pass over the parse tree, not a fancier parser, precisely because climbing to Type 1 is too expensive to use directly.

4.5 Type 0: the limits every compiler runs into

A compiler is just a program, and the programs it analyses are Turing-complete. That is the outermost, least-decidable level. The Halting Problem is undecidable, and by Rice's Theorem so is essentially every nontrivial semantic property of a program:

- *Does this loop always terminate?* Undecidable (it is the Halting Problem).
- *Is this code ever executed?* Undecidable: exact dead-code detection would solve Halting.
- *Do these two functions compute the same result?* Undecidable (program equivalence).

Real optimisers therefore never try to answer these exactly. They compute a **sound, conservative approximation**: “maybe reachable” rather than “definitely unreachable”, never the other way around. An unsound answer would let the optimiser delete live code, which violates correctness, the one property we refuse to trade.

In the real world. This is why `-Wunreachable-code` sometimes misses dead code and why the optimiser sometimes keeps a check you “know” is redundant. Both are conservative approximations, on purpose.

4.6 Putting it together

Hierarchy level	Machine	Compiler phase	Deciding membership
Type 3 Regular	Finite automaton	Lexical analysis	Yes, $O(n)$
Type 2 Context-free	Pushdown automaton	Syntax analysis	Yes, $O(n)$ to $O(n^3)$
Type 1 Context-sensitive	Linear-bounded automaton	Semantic analysis (by hand, not by grammar)	Yes, PSPACE-complete
Type 0 Rec. enumerable	Turing machine	The compiler itself; whole-program analysis	Not in general

Summary

The further out in the hierarchy, the more powerful the machine and the less you can decide about it. Good compiler design pushes each problem to the *cheapest* level that can handle it: regular for tokens, deterministic context-free for syntax, a hand-written tree walk for the context-sensitive bits, and conservative approximation for everything undecidable.

5 One More Design Decision: The Implementation Language

Traditionally many compilers were **self-hosting**: written in the language they compile. Like pulling yourself up by your own bootstraps, it needs a first compiler written in something else, but it has real benefits: fewer external dependencies, and it exposes weaknesses in the language design quickly. Today’s languages make cross-language implementation easy, so self-hosting matters less.

What actually matters when choosing:

- **Modularity and static types.** A compiler is a large, multi-person project with many phases sharing interfaces; a static type system guides interface design.

History. The first self-hosting compiler was Hart and Levin’s Lisp compiler (1962). C’s compiler was self-hosting by 1973; Rust’s became self-hosting in 2011, replacing an OCaml bootstrap.

- **Garbage collection.** Compilers build complex, often cyclic data structures (ASTs, control-flow graphs) shared across phases. Manual memory management complicates every phase; GC is recommended.
- **Pattern matching.** Functional languages with algebraic data types suit the tree-structured front end beautifully, but arbitrary graph structures in the back end can be harder to manage there.

6 Tricks, Techniques and Classic Problems

Debugging: Tools for looking inside the pipeline

- `gcc -E` (or `clang -E`): stop after preprocessing. `-S`: stop at assembly. `-c`: stop at object code.
- `gcc -fdump-tree-all` / `clang -Xclang -ast-dump`: print the AST. `clang -S -emit-llvm`: print the IR.
- `objdump -d` and `nm`: inspect object code and its symbol table. `readelf -s` for symbols in ELF.
- `gdb` with `layout asm` or `disassemble`: single-step the target code while watching registers (info registers).
- godbolt.org: interactive source-to-assembly view for dozens of compilers.

Technique: Reading optimised assembly without panic

1. Find the loop first: a backward jump (`jne L1` where `L1` is above) marks it.
2. Identify what each register *means* at loop entry (here `rcx=n`, `rdx=i`, `rdi=a`).
3. Expect idioms: `xor r,r` is “zero”, `lea` is arithmetic, `cmov` is a branchless `if`, `test` is a compare against zero.
4. Trust the comparison direction less than the source: optimisers rewrite `<` as `!=` when they can prove monotonicity.

Warning / Gotcha: Common student mistakes

- **“The compiler produces machine code.”** Usually it produces assembly *text*; the assembler produces bytes and the linker produces the executable.
- **Confusing compile-time and link-time errors.** Type errors and undeclared names are the compiler’s. “Undefined reference” is the linker’s.
- **Thinking semantic analysis is part of parsing.** It is a separate tree walk because the constraints are context-sensitive.
- **Expecting the optimiser to be exact.** It cannot be (Rice). It is sound and conservative, so it sometimes keeps code you think is dead.
- **“Interpreters are always slower, so compilers are always better.”** For a one-off script the compile time dominates; interpret it.

Worked example: Classic exam and interview questions

Q1. Why does an m -language, n -target compiler suite need only $m + n$ components rather than $m \times n$?

Because the front ends all emit one machine-independent IR and the back ends all consume it; the IR is the shared contract.

Q2. Java is “compiled” and Python is “interpreted”. Is this accurate?

Only loosely. `javac` compiles to bytecode, which the JVM first interprets and then JIT-compiles. CPython compiles source to bytecode too, then interprets it. The real difference is whether native code is generated.

Q3. Give a property of programs that a compiler cannot decide in general, and say what it does instead.

Whether a statement is ever executed (reachability). Deciding it exactly would decide Halting. The compiler computes a conservative over-approximation of the reachable set: it deletes code only when it can *prove* it unreachable.

Q4. Which phase discards parentheses, and why is that safe?

Parsing. Parentheses only guide the parser to the right tree shape; once the AST encodes the shape, they carry no further information.

Q5. In the optimised `findIndex`, why is `mov rdx, rax` placed before `jne L1` rather than after?

It is needed on the loop-back path (`i = i+1`) and harmless on the fall-through path (where `rax` is decremented and returned, and `rdx` is dead). Hoisting it above the branch saves a jump target and an instruction on the hot path.

Q6. Why is lexing $O(n)$ but general context-free parsing $O(n^3)$?

A DFA does constant work per character. A general CFG may require exploring many derivations per position; CYK’s triple loop over span start, span end, and split point gives $O(n^3)$. LL/LR parsers regain $O(n)$ by restricting the grammar to a deterministic subset.

Summary: Chapter 1 in six lines

1. A compiler translates source to target, preserving meaning; correctness beats speed.
2. Compile when you will run many times; interpret when once.
3. Phases: `lex` $\rightarrow$ `parse` $\rightarrow$ `check` $\rightarrow$ `generate` $\rightarrow$ `optimise`, each with its own intermediate language.
4. Register allocation is the most important optimisation: it turns 18 memory accesses into zero.
5. Lexing is regular, parsing is deterministic context-free, checking is a separate tree walk, and whole-program questions are undecidable, so optimisers approximate conservatively.
6. Front end (machine-independent) + IR + back end (language-independent); then assembler, linker, loader.